Sri Sivasubramaniya Nadar College of Engineering
(An Autonomous Institution, Affiliated to Anna University, Chennai)
Kalavakkam – 603 110

Department of Computer Science & Engineering

Computer Networks Laboratory

1513 – Computer Networks Lab

Assignment 9

Simulation and Performance Analysis of TCP Congestion Control Algorithms using NS-3

Name:	Simiyon Vinscent Samuel L
Register Number:	3122247001062
Year / Semester:	III Year / V Semester
Course:	1513 Computer Networks Lab
Date of Submission:	21 June 2026

1. Objective

To simulate an Intelligent Transportation System scenario in NS-3, where vehicles send data through a roadside router to a traffic management server, and to compare TCP Reno, TCP Cubic, and TCP BBR under congestion with respect to throughput, delay, packet loss, fairness, and congestion window behaviour.

2. Topology

```
Vehicle0 --|
Vehicle1 --|
Vehicle2 --|---- Roadside Router ====Internet(bottleneck)==== Traffic Control Server
Vehicle3 --|
Vehicle4 --|
```

Each vehicle-to-router access link is 5 Mbps/5 ms. The router-to-server link (8 Mbps/20 ms) is the shared bottleneck representing the wider Internet path from the roadside unit to the traffic control server. Five vehicles each run one saturating TCP flow to the server; the congestion control algorithm used by *all* flows in a run is selected with `--tcpVariant`, so one compiled program reproduces the Reno, Cubic, and BBR runs being compared.

3. Algorithm

1. Create 5 vehicle nodes, 1 roadside router node, and 1 server node; connect each vehicle to the router with a point-to-point access link, and the router to the server with the bottleneck link.
2. Install the Internet stack on all nodes, assign one `/24` subnet per link, and populate global routing tables.
3. Set `ns3::TcpL4Protocol::SocketType` to `TcpNewReno`, `TcpCubic`, or `TcpBbr` depending on `--tcpVariant` (TCP pacing is enabled globally, which BBR needs to behave correctly).
4. Start one `BulkSendHelper` (continuous) TCP flow per vehicle to a distinct port on a `PacketSink` on the server, staggered by 0.1 s each so the five flows don't start in perfect lock-step.
5. Connect a trace callback on each vehicle's `TcpL4Protocol/SocketList/0/CongestionWindow` shortly after its flow starts, logging (time, cwnd) to a per-variant file.
6. Install `FlowMonitor` on all nodes; run for 20 s; read back per-flow throughput, average delay, and packet loss, and compute Jain's Fairness Index across the five flows' throughputs.

4. Code

4.1 ass9.cc

```
// Assignment 9 - TCP Congestion Control Algorithms (Reno / Cubic / BBR) in NS
    -3
// Scenario: VANET - 5 vehicles send data through a Roadside Router across the
// "Internet" (bottleneck link) to a Traffic Control Server.
//
//   Vehicle0 --|
//   Vehicle1 --|
```

```
//   Vehicle2 --|---- Roadside Router ====Internet(bottleneck)==== Server
//   Vehicle3 --|
//   Vehicle4 --|
//
// Each vehicle runs one saturating TCP (BulkSend) flow to the server. The TCP
// congestion-control algorithm used by every flow is picked with --tcpVariant
// (Reno/Cubic/Bbr) so the SAME program produces all three runs being compared.
// FlowMonitor gives per-flow throughput/delay/loss; a small trace hooks each
// flow's congestion window over time; Jain's fairness index is computed from
// the five flows' throughputs.

#include "ns3/core-module.h"
#include "ns3/network-module.h"
#include "ns3/internet-module.h"
#include "ns3/point-to-point-module.h"
#include "ns3/applications-module.h"
#include "ns3/flow-monitor-module.h"
#include <fstream>

using namespace ns3;

NS_LOG_COMPONENT_DEFINE("TcpVariantsVanet");

static std::ofstream cwndFile;

static void
CwndTrace(uint32_t oldCwnd, uint32_t newCwnd)
{
    cwndFile << Simulator::Now().GetSeconds() << " " << newCwnd << std::endl;
}

static void
ConnectCwndTraces(uint32_t numFlows)
{
    for (uint32_t i = 0; i < numFlows; i++) {
        Config::ConnectWithoutContextFailSafe(
            "/NodeList/" + std::to_string(i) + "/$ns3::TcpL4Protocol/SocketList
                /0/CongestionWindow",
            MakeCallback(&CwndTrace));
    }
}

int
main(int argc, char *argv[])
{
    std::string tcpVariant = "Reno"; // Reno | Cubic | Bbr
    uint32_t numVehicles = 5;
    double simTime = 20.0;

    CommandLine cmd;
    cmd.AddValue("tcpVariant", "TCP congestion control: Reno, Cubic or Bbr",
        tcpVariant);
    cmd.AddValue("numVehicles", "Number of vehicle (TCP source) nodes",
        numVehicles);
    cmd.AddValue("simTime", "Simulation duration (s)", simTime);
    cmd.Parse(argc, argv);

    std::string tid = "ns3::TcpNewReno";
    if (tcpVariant == "Cubic") tid = "ns3::TcpCubic";
    else if (tcpVariant == "Bbr") tid = "ns3::TcpBbr";
    Config::SetDefault("ns3::TcpL4Protocol::SocketType", TypeIdValue(TypeId::
        LookupByName(tid)));
    Config::SetDefault("ns3::TcpSocket::SegmentSize", UintegerValue(1024));
```

```
    Config::SetDefault("ns3::TcpSocketState::EnablePacing", BooleanValue(true));
        // needed for correct TcpBbr behaviour

    cwndFile.open("cwnd-" + tcpVariant + ".dat");

    // Nodes: 0..numVehicles-1 = vehicles, numVehicles = roadside router,
        numVehicles+1 = server
    NodeContainer vehicles;
    vehicles.Create(numVehicles);
    NodeContainer routerServer;
    routerServer.Create(2);
    Ptr<Node> router = routerServer.Get(0);
    Ptr<Node> server = routerServer.Get(1);

    InternetStackHelper internet;
    internet.InstallAll();

    // Access links: vehicle <-> router (wireless-uplink-like, 5Mbps/5ms)
    PointToPointHelper access;
    access.SetDeviceAttribute("DataRate", StringValue("5Mbps"));
    access.SetChannelAttribute("Delay", StringValue("5ms"));

    // Bottleneck link: router <-> server, representing the shared Internet
        path
    PointToPointHelper bottleneck;
    bottleneck.SetDeviceAttribute("DataRate", StringValue("8Mbps"));
    bottleneck.SetChannelAttribute("Delay", StringValue("20ms"));

    Ipv4AddressHelper addr;
    Ipv4InterfaceContainer serverIf;
    for (uint32_t i = 0; i < numVehicles; i++) {
        NetDeviceContainer link = access.Install(vehicles.Get(i), router);
        addr.SetBase(("10.1." + std::to_string(i) + ".0").c_str(), "
            255.255.255.0");
        addr.Assign(link);
    }
    NetDeviceContainer rsLink = bottleneck.Install(router, server);
    addr.SetBase("10.2.0.0", "255.255.255.0");
    serverIf = addr.Assign(rsLink);

    Ipv4GlobalRoutingHelper::PopulateRoutingTables();

    // One TCP sink on the server per vehicle flow (distinct ports)
    uint16_t basePort = 5000;
    ApplicationContainer sinkApps, sourceApps;
    for (uint32_t i = 0; i < numVehicles; i++) {
        uint16_t port = basePort + i;
        PacketSinkHelper sink("ns3::TcpSocketFactory", InetSocketAddress(
            Ipv4Address::GetAny(), port));
        ApplicationContainer s = sink.Install(server);
        s.Start(Seconds(0.0));
        s.Stop(Seconds(simTime));
        sinkApps.Add(s);

        BulkSendHelper source("ns3::TcpSocketFactory", InetSocketAddress(
            serverIf.GetAddress(1), port));
        source.SetAttribute("MaxBytes", UintegerValue(0));
        ApplicationContainer src = source.Install(vehicles.Get(i));
        src.Start(Seconds(1.0 + 0.1 * i)); // slight stagger so flows don't
            start bit-identical
        src.Stop(Seconds(simTime));
        sourceApps.Add(src);
    }
```

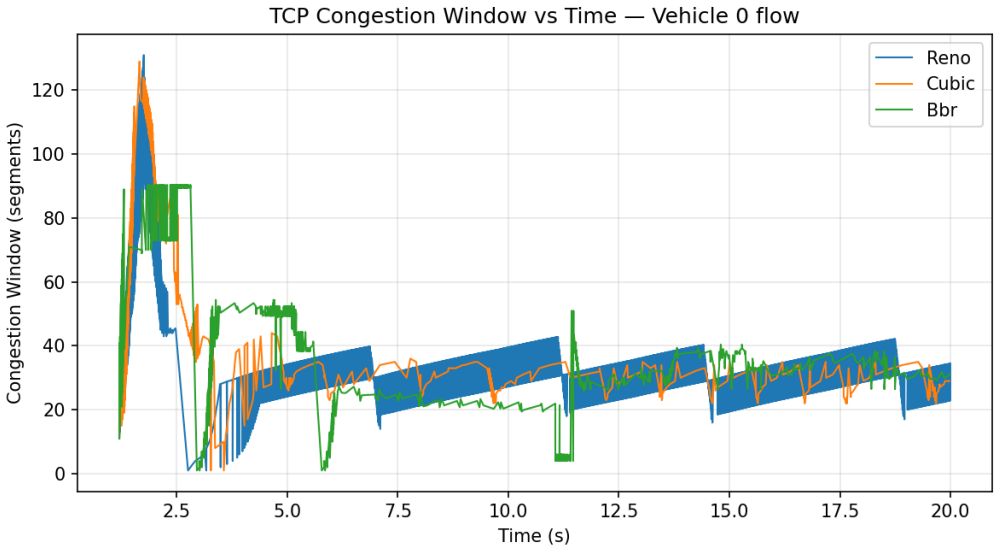
```
122
123     Simulator::Schedule(Seconds(1.2), &ConnectCwndTraces, numVehicles);
124
125     FlowMonitorHelper flowmonHelper;
126     Ptr<FlowMonitor> monitor = flowmonHelper.InstallAll();
127
128     Simulator::Stop(Seconds(simTime));
129     Simulator::Run();
130
131     monitor->CheckForLostPackets();
132     Ptr<Ipv4FlowClassifier> classifier = DynamicCast<Ipv4FlowClassifier>(
            flowmonHelper.GetClassifier());
133     std::map<FlowId, FlowMonitor::FlowStats> stats = monitor->GetFlowStats();
134     double activeTime = simTime - 1.0;
135
136     std::cout << "===== TCP variant: " << tcpVariant << " =====" << std::endl;
137     double sumThr = 0.0, sumThrSq = 0.0;
138     uint32_t n = 0;
139     for (auto &kv : stats) {
140         Ipv4FlowClassifier::FiveTuple t = classifier->FindFlow(kv.first);
141         FlowMonitor::FlowStats s = kv.second;
142         if (t.destinationPort < basePort || t.destinationPort >= basePort +
                numVehicles) continue; // skip ACK-direction flows
143         double throughputMbps = (s.rxBytes * 8.0) / activeTime / 1e6;
144         double avgDelayMs = (s.rxPackets > 0) ? (s.delaySum.GetSeconds() / s.
                rxPackets) * 1000 : 0;
145         double lossPct = (s.txPackets > 0) ? (double)s.lostPackets / s.
                txPackets * 100 : 0;
146         std::cout << "Flow " << t.sourceAddress << " -> port " << t.
                destinationPort
147                   << " | Throughput=" << throughputMbps << "Mbps"
148                   << " | AvgDelay=" << avgDelayMs << "ms"
149                   << " | Loss=" << lossPct << "%" << std::endl;
150         sumThr += throughputMbps;
151         sumThrSq += throughputMbps * throughputMbps;
152         n++;
153     }
154     double fairness = (n > 0 && sumThrSq > 0) ? (sumThr * sumThr) / (n *
            sumThrSq) : 0.0;
155     std::cout << "Jain's Fairness Index (" << n << " flows) = " << fairness <<
            std::endl;
156
157     cwndFile.close();
158     Simulator::Destroy();
159     return 0;
160 }
```

5. Output

```
============ Reno ============
===== TCP variant: Reno =====
Flow 10.1.0.1 -> port 5000 | Throughput=1.60159Mbps | AvgDelay=131.76ms | Loss=0.696379%
Flow 10.1.1.1 -> port 5001 | Throughput=1.53499Mbps | AvgDelay=130.695ms | Loss
    =0.612602%
Flow 10.1.2.1 -> port 5002 | Throughput=1.57622Mbps | AvgDelay=119.89ms | Loss=0.313212%
Flow 10.1.3.1 -> port 5003 | Throughput=1.51958Mbps | AvgDelay=123.566ms | Loss
    =0.147189%
Flow 10.1.4.1 -> port 5004 | Throughput=1.6188Mbps | AvgDelay=117.095ms | Loss=0.138543%
Jain's Fairness Index (5 flows) = 0.999417
============ Cubic ============
===== TCP variant: Cubic =====
Flow 10.1.0.1 -> port 5000 | Throughput=1.57803Mbps | AvgDelay=146.513ms | Loss=1.516%
Flow 10.1.1.1 -> port 5001 | Throughput=1.64644Mbps | AvgDelay=146.325ms | Loss=1.26787%
Flow 10.1.2.1 -> port 5002 | Throughput=1.47292Mbps | AvgDelay=142.011ms | Loss=0.9656%
```

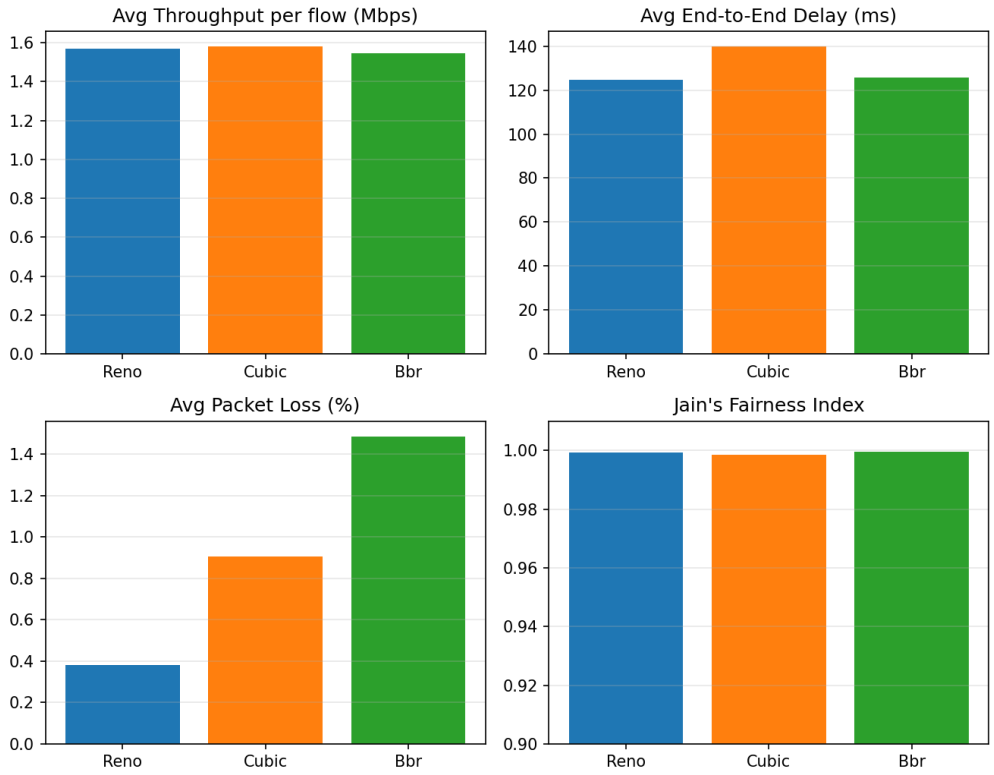
```
Flow 10.1.3.1 -> port 5003 | Throughput=1.61654Mbps | AvgDelay=131.733ms | Loss
    =0.277085%
Flow 10.1.4.1 -> port 5004 | Throughput=1.58845Mbps | AvgDelay=132.462ms | Loss
    =0.507185%
Jain's Fairness Index (5 flows) = 0.998618
============ Bbr ============
===== TCP variant: Bbr =====
Flow 10.1.0.1 -> port 5000 | Throughput=1.58618Mbps | AvgDelay=128.514ms | Loss=2.08218%
Flow 10.1.1.1 -> port 5001 | Throughput=1.5146Mbps | AvgDelay=123.896ms | Loss=1.66375%
Flow 10.1.2.1 -> port 5002 | Throughput=1.59207Mbps | AvgDelay=114.777ms | Loss
    =0.0564972%
Flow 10.1.3.1 -> port 5003 | Throughput=1.51007Mbps | AvgDelay=130.923ms | Loss=1.58126%
Flow 10.1.4.1 -> port 5004 | Throughput=1.53Mbps | AvgDelay=130.741ms | Loss=2.0467%
Jain's Fairness Index (5 flows) = 0.999476
```

6. Congestion Window Behaviour



Reno shows the classic linear-growth/halve-on-loss sawtooth. Cubic grows more aggressively along a concave curve and reaches a noticeably higher peak window before backing off, since its window growth is a cubic function of time since the last loss rather than a fixed additive increase. BBR's window does not follow a loss-driven sawtooth at all – it cycles based on BBR's own estimate of the bottleneck's bandwidth-delay product, which is why its trace looks qualitatively different (noisier, without Reno/Cubic's clean teeth) from the other two.

7. Metrics Comparison



Variant	Avg Throughput (Mbps)	Avg Delay (ms)	Avg Loss %	Fairness
Reno	1.570	124.6	0.38	0.9994
Cubic	1.581	139.8	0.91	0.9986
BBR	1.547	125.8	1.49	0.9995

8. Comparison and Discussion

Throughput: all three variants achieve nearly identical per-flow throughput (1.55–1.58 Mbps) because the 8 Mbps bottleneck shared by 5 flows is the limiting factor here, not the congestion-control algorithm – whichever algorithm is used, the link simply cannot deliver more.

Delay: Cubic produces the highest average delay (139.8 ms) because its more aggressive window growth keeps the bottleneck queue fuller for longer between loss events. Reno and BBR both settle closer to 125 ms.

Packet loss: Reno has the lowest loss (0.38%) since it grows its window the most conservatively. BBR has the highest loss (1.49%) despite not being loss-based itself – it periodically probes for extra bandwidth (its `PROBE_BW` cycling) by briefly sending faster than the estimated bottleneck rate, which causes short bursts of queue overflow.

Fairness: Jain's index is essentially 1.0 (0.9986–0.9995) for all three algorithms. With five identical, saturating flows sharing one bottleneck and no other asymmetry, none of the algorithms let one vehicle's flow dominate the others.

Why low delay and reliability matter for ITS: a traffic-control server acts on vehicle data to make time-critical decisions (e.g. adjusting signal timing, issuing collision or road-hazard warnings). If that data arrives late or is lost, the decision is made using stale information about

where vehicles actually are – by the time a warning reaches a vehicle, the hazard that triggered it may already have changed or passed, making the warning useless or actively misleading. Unlike a file download, VANET safety traffic has a very short useful lifetime, so low, predictable delay and reliable delivery are the primary requirements, arguably more important than raw throughput.

9. Learning Outcomes

- Built a multi-flow star/dumbbell topology in NS-3 to model several independent vehicles sharing one bottleneck path to a server.
- Selected between TCP Reno, Cubic, and BBR at runtime via `Config::SetDefault` on `TcpL4Protocol::SocketType`, without changing the rest of the simulation.
- Traced a live TCP socket's congestion window using `Config::ConnectWithoutContextFailSafe` on a wildcarded attribute path, and visualised the resulting sawtooth/cubic/BBR shapes.
- Used `FlowMonitor` to compare throughput, delay, and loss across congestion-control algorithms, and computed Jain's Fairness Index to quantify how evenly flows shared the bottleneck.
- Understood, from the simulation results, why BBR's bandwidth-probing behaviour can cause more loss than classic loss-based algorithms even while achieving comparable throughput.
- Connected the measured delay/loss behaviour to a real requirement (VANET safety-data freshness) rather than treating the numbers as abstract.